CT-378 Machine Learning — Semester Project Guidelines

Overview and timeline

Each group solves one real problem end to end with ML and deploys it three ways. Worth **10 sessional marks**, final demonstration in **Week 15**. Groups of **2–3 students**; every member must contribute code and answer viva questions on any part.

Milestone	Week	What to submit
Proposal	12	Half-page: problem, data source, target variable, success metric
Final demo, report, viva	15	All deliverables below

Problem and data

- **Problem:** a classification, regression, clustering or time-series task with a clear user (e.g. crop yield, load-shedding demand, fake-review detection). No duplicate topics across groups; first come approval.
- **Not allowed:** Iris, Titanic, MNIST, Boston Housing or any dataset used.
- **Data:** collect it yourselves (survey, sensors, web scraping within site terms, public APIs) or combine at least two public sources. Minimum **5,000 rows and 15 features**.
- Document the source, collection method, licence and date for every dataset. Remove personal identifiers.

ML lifecycle (all steps required)

1. **EDA:** distributions, missing values, outliers, correlations, class balance (at least 5 plots). 2. **Pre-processing:** imputation, encoding, scaling, feature engineering, all inside a scikit-learn `Pipeline` fitted on training data only (no data leakage).
3. **Split:** train/validation/test (e.g. 70/15/15); time-ordered split for time series. 4. **Modelling:** a simple baseline plus **at least three** algorithms from the course (e.g. KNN, Logistic/Linear Regression, SVM, Random Forest, Boosting, etc).
5. **Tuning:** 5-fold cross-validation with GridSearchCV or RandomizedSearchCV. 6. **Evaluation:** metrics fit to the task (F1/ROC-AUC/confusion matrix, or MAE/RMSE/R^2), reported once on the held-out test set; discuss over/under-fitting and errors.
7. **Save:** final pipeline with `joblib`; pin versions in `requirements.txt`.

Deployment (choose any one of the following options)

Platform	Requirement
Streamlit	Input form, prediction with confidence/probability, one explanatory chart; deployed on Streamlit Community Cloud with a public URL

Platform	Requirement
Hugging Face	Model card and pipeline on the HF Hub, plus a live Space (Gradio or Streamlit)
Local web app	Flask or FastAPI backend with a `/predict` endpoint and an HTML front end, run on `localhost` during the demo; input validation and error messages required

Deliverables

- **GitHub repo:** `data/` (or download script), `notebooks/`, `app/`, `models/`, `requirements.txt`, and a README with setup steps and all three deployment links.
- **Report (PDF, 3–4 pages):** Abstract · Problem and motivation · Data collection · EDA · Methodology · Results (comparison table + plots) · Deployment (screenshots) · Limitations and future work · References · Individual contribution table.
- **Demo:** 10 minutes per group: 3 min problem and data, 4 min live demo of the apps, 3 min results; then viva.

Marking scheme (10 marks)

Component	Marks	Assessed on
Demonstration	4	ML lifecycle quality (2); all three deployments working live (2)
Report	3	Completeness, correct metrics, clear figures, proper citations
Viva (individual)	3	Each member explains their code, model choices and results

Rules: viva marks are individual, so group members may score differently. A deployment that fails live scores zero for that platform. Copied code, reports or datasets without citation get zero for the whole project. Late submission loses 1 mark per day.